

School of Electronics and Communication Engineering

Report on:

WiFi Based Multi-Sensor Rover with Real-time Video Streaming and Mobile Control Interface

TITLE	WiFi Based Multi-Sensor Rover with Real-time Video Streaming and Mobile control Interface
LEVEL OF PROJECT	
FACULTY	Dr. Jayakumar S

STUDENT DETAIL	NAME	REG.NO	SCHOOL
1	Aayushi	25BML0022	SENSE
	Asutosh Aum	25BEC0275	SENSE
	Rishi Senthil	25BEC0336	SENSE
	Saswin Manivel K	25BME0331	SMEC

Signature of the Faculty	HoD Signature	Signature of the Dean
--------------------------	---------------	-----------------------

ABSTRACT

The rapid advancement of robotics and Internet of Things (IoT) technologies has enabled the development of intelligent systems for remote monitoring and control. This project focuses on the design and implementation of a WiFi-based multi-sensor rover capable of real-time video streaming and mobile-based control. The rover is developed using a microcontroller-based architecture integrated with a WiFi communication module, enabling seamless wireless connectivity between the user and the system.

The rover is equipped with multiple sensors, including ultrasonic sensors for obstacle detection, temperature and humidity sensors for environmental monitoring, and gas sensors for detecting hazardous conditions. These sensors continuously collect data, which is processed and transmitted to the user interface. A camera module is mounted on the rover to provide live video streaming, allowing the user to visually monitor the surroundings and navigate effectively.

The mobile control interface is designed to provide a user-friendly platform for controlling the rover's movement in multiple directions while simultaneously displaying real-time sensor data and video feed. The system ensures low latency communication and efficient data transmission over WiFi networks.

This project emphasizes modular design, energy efficiency, and scalability, making it adaptable for various real-world applications such as military surveillance, search and rescue operations, industrial inspection, and smart environmental monitoring. The implementation demonstrates the integration of embedded systems, wireless communication, and real-time data processing, contributing to the development of intelligent robotic systems for modern applications.

INTRODUCTION

In recent years, the integration of robotics with wireless communication and Internet of Things (IoT) technologies has significantly transformed the way remote monitoring and control systems are designed. Autonomous and semi-autonomous robotic systems are increasingly being used in environments that are hazardous, inaccessible, or unsafe for human presence. This has created a growing demand for intelligent rovers capable of real-time data acquisition, live video transmission, and remote operation.

The WiFi-based multi-sensor rover is developed as a versatile solution to address these challenges. It combines mobility, sensing capabilities, and wireless communication into a single platform. The rover is equipped with multiple sensors to monitor environmental parameters such as obstacles, temperature, humidity, and harmful gases. These sensors enable the system to gather critical real-time data from its surroundings.

A key feature of this system is real-time video streaming, achieved using an onboard camera module. This allows users to visually monitor the rover's path and surroundings, enhancing navigation and decision-making. The rover is controlled through a mobile interface, which provides a user-friendly means of directing its movement and accessing live sensor data and video feed via WiFi connectivity.

The proposed system is designed with a focus on cost-effectiveness, flexibility, and scalability. It can be applied in a wide range of fields including surveillance, disaster management, industrial inspection, and environmental monitoring. By integrating embedded systems, wireless networking, and sensor technologies,

this project demonstrates a practical approach to building an efficient and intelligent robotic platform.

OBJECTIVES

- To ensure low-cost and energy-efficient system design suitable for practical applications.
- To demonstrate the integration of embedded systems, IoT, and robotics in a single platform.
- To design and develop a WiFi-based rover capable of remote operation using a mobile control interface.
- To integrate multiple sensors (such as ultrasonic, temperature, humidity, and gas sensors) for real-time environmental monitoring.
- To implement real-time video streaming using an onboard camera for enhanced navigation and surveillance.
- To establish reliable wireless communication between the rover and the user through WiFi technology.
- To develop a user-friendly mobile interface for controlling the rover and viewing live sensor data and video feed.
- To enable obstacle detection and avoidance for safe and efficient movement of the rover.
- To explore real-world applications such as disaster management, security surveillance, and industrial inspection.

SYSTEM ARCHITECTURE AND WORKFLOW

The WiFi-based multi-sensor rover system follows a layered and modular architecture consisting of hardware and software subsystems that work together to achieve real-time monitoring, control, and data transmission. The architecture can be broadly divided into five major layers: sensing layer, processing layer, communication layer, actuation layer, and user interface layer.

Sensing Layer

The sensing layer comprises multiple sensors mounted on the rover to collect environmental data. An ultrasonic sensor is used for obstacle detection and distance measurement, enabling safe navigation. A temperature and humidity sensor monitors environmental conditions, while a gas sensor detects the presence of harmful or toxic gases. These sensors continuously generate analog or digital signals based on surrounding conditions.

Processing Layer

The processing layer is handled by a microcontroller such as an ESP32 or Arduino. This unit acts as the central controller, responsible for acquiring sensor data, processing it, and making decisions. It converts analog signals into digital data (if required), applies necessary logic, and prepares the data for transmission. It also interprets commands received from the user interface and controls the rover's movement accordingly.

Communication Layer

The communication layer is implemented using WiFi technology, enabling bidirectional data transfer between the rover and the mobile device. The microcontroller either uses an inbuilt WiFi module (ESP32) or an external module (ESP8266) to establish a connection. Sensor data and video streams are transmitted to the user, while control commands are received from the mobile interface. Communication protocols such as HTTP or WebSocket may be used to ensure reliable and low-latency transmission.

Actuation Layer

The actuation layer includes motor drivers (such as L298N) and DC motors that control the movement of the rover. Based on commands received from the processing unit, the motor driver regulates the speed and direction of the motors, allowing the rover to move forward, backward, and turn in different directions. This layer ensures precise motion control and responsiveness.

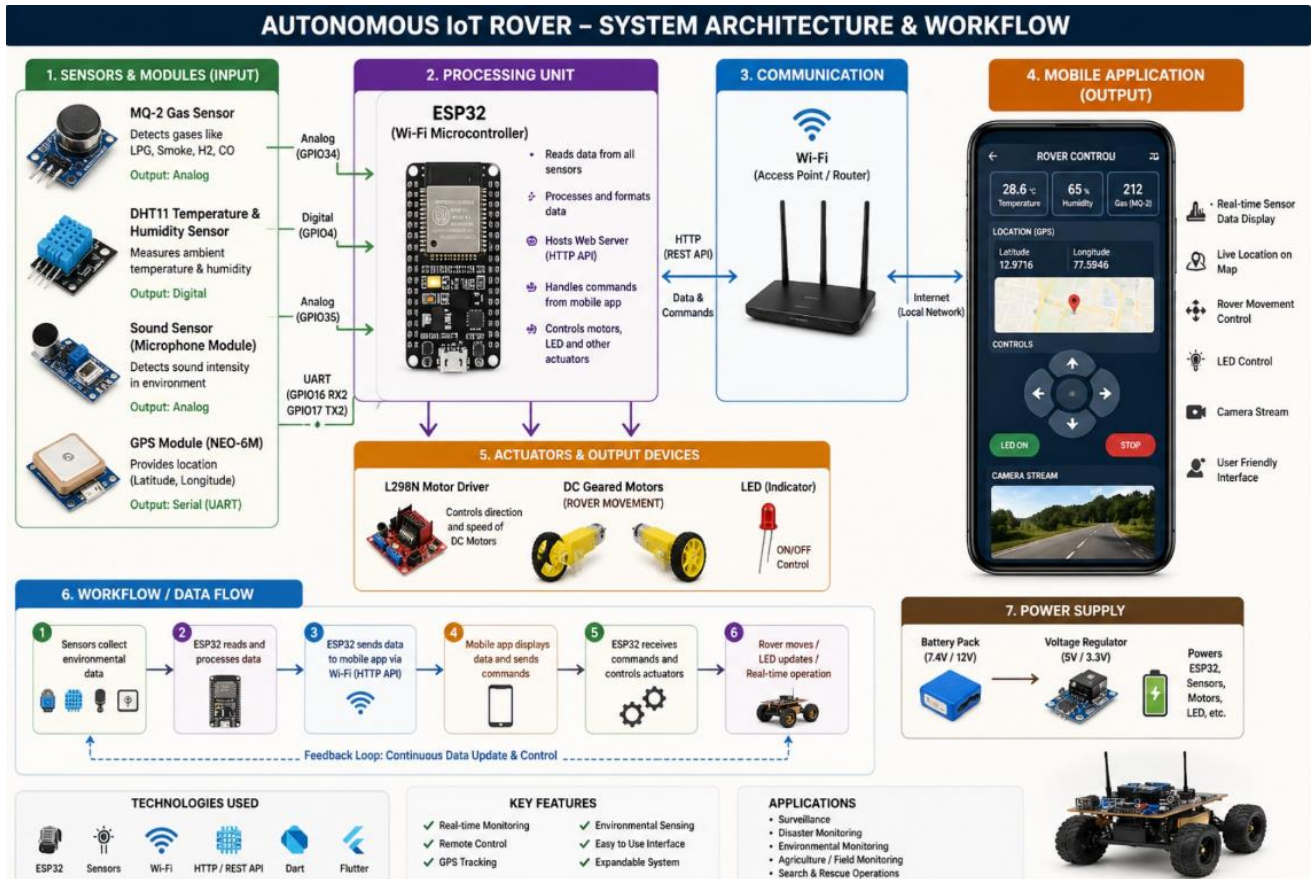
Visual Streaming Subsystem

The visual streaming subsystem consists of a camera module (such as ESP32-CAM) that captures real-time video. The video is encoded and transmitted over the WiFi network to the user interface. This feature provides live visual feedback, which is crucial for remote navigation and surveillance.

User Interface Layer

The user interface layer is developed as a mobile application or web-based dashboard. It provides control buttons for navigation and displays real-time sensor data and live video feed. The interface communicates with the rover through the WiFi network, ensuring seamless interaction between the user and the system.

The entire system operates in a real-time loop, where sensing, processing, communication, and actuation occur simultaneously. The modular design ensures easy scalability, allowing additional sensors or functionalities to be integrated without major modifications.



WORKING PRINCIPLE

Parallel Communication Channels in System Architecture

The proposed WiFi-based multi-sensor rover operates using three independent yet parallel communication channels, namely the control channel, sensor data channel, and video streaming channel. This separation ensures efficient data handling, reduced latency, and reliable real-time performance.

1. Control Channel (Mobile Application → ESP32)

The control channel is responsible for transmitting user commands from the mobile interface to the rover for navigation and movement control. A mobile application developed using the Flutter framework provides a joystick-based control mechanism. The joystick widget captures the user's touch input and computes the corresponding X and Y displacement values.

These displacement values are processed and mapped into specific motion commands such as forward, backward, left, right, and speed variations. The commands are then encoded into HTTP GET requests in the format:

```
GET /move?dir=forward&speed=180
```

These requests are transmitted over a local WiFi network to the ESP32 microcontroller. The ESP32 hosts a web server using its built-in WebServer library, typically listening on port 80. Within its main execution loop, the ESP32 continuously monitors incoming HTTP requests, parses the command parameters (direction and speed), and translates them into appropriate Pulse Width Modulation (PWM) signals.

These PWM signals are sent to the motor driver module (such as L298N), which in turn controls the speed and direction of the DC motors. Due to the use of a local WiFi network, the round-trip latency of control signals is typically maintained below 50 milliseconds, ensuring smooth and responsive rover operation.

2. Sensor Data Channel (ESP32 → Mobile Application)

The sensor data channel facilitates the transmission of environmental data collected by the rover back to the user interface. The ESP32 is interfaced with multiple sensors, including the MQ2 gas sensor, DHT11 temperature and humidity sensor, and a GPS module for location tracking.

On the mobile application side, a background timer periodically triggers HTTP GET requests to the ESP32 endpoint:

```
GET /sensors
```

This polling mechanism typically operates at intervals of 1 to 2 seconds. Upon receiving the request, the ESP32 reads real-time data from all connected sensors. The collected data is then structured into a JSON (JavaScript Object Notation) format for efficient transmission. An example response is:

```
{ "temp": 32,  
  "humidity": 65,  
  "gas": 120,  
  "lat": 12.97,  
  "lng": 77.59  
}
```

This JSON object is sent back as the HTTP response body. On the Flutter side, the built-in `dart:convert` library is used to parse the JSON data. The parsed values are then used to dynamically update the user interface, allowing real-time visualization of environmental parameters.

3. Video Streaming Channel (ESP32-CAM → Mobile Application)

The video streaming channel operates independently of the main control and sensor communication channels. It is handled by a dedicated ESP32-CAM module, which is responsible for capturing and transmitting live video.

The ESP32-CAM runs preloaded firmware such as `CameraWebServer`, which hosts a video streaming server. It provides a Motion JPEG (MJPEG) stream accessible via a specific IP address and port, typically:

```
http://192.168.1.101:81/stream
```

This MJPEG stream consists of a continuous sequence of JPEG images transmitted over HTTP in a multipart format. On the mobile application side, this stream is rendered using components such as a `WebView` or an `Image.network` widget in Flutter. These components continuously fetch and display the incoming frames, creating a real-time video effect.

Since this channel operates independently, it does not interfere with control or sensor data transmission, thereby improving overall system performance and reducing bottlenecks.

HARDWARE COMPONENTS

ESP32 (Main Controller)

The ESP32 is the brain of the rover. It is a dual-core 32-bit microcontroller running at 240 MHz with built-in WiFi (802.11 b/g/n) and Bluetooth. Its key role here is to host a lightweight HTTP web server (using the AsyncWebServer or WebServer library), process incoming control commands, read all connected sensors, and output PWM signals to the motor driver. It has 36 GPIO pins, multiple UART interfaces (for GPS), ADC channels (for MQ2), and supports I2C/SPI. It operates at 3.3V logic with 5V-tolerant inputs on select pins.

ESP32-CAM

A variant of the ESP32 with an integrated OV2640 camera module (2 megapixels) and a microSD card slot. It runs a dedicated camera server firmware that captures frames from the camera and encodes them as MJPEG, broadcasting the stream over its own WiFi IP address. Since it handles video independently, it doesn't burden the main ESP32 with image processing. It has limited GPIO (only a few pins accessible), so it's used exclusively for video. It requires a separate 5V power supply and a USB-to-TTL adapter for programming (no onboard USB chip).

Motor Driver (L298N or L293D)

The motor driver is an H-bridge IC that allows the low-current GPIO pins of the ESP32 (max ~12mA per pin) to drive high-current DC motors (up to 2A per channel for L298N). The ESP32 sends two direction signals (IN1, IN2) per motor and a PWM signal to the Enable pin to control speed. The L298N has two independent H-bridge channels, so it can control two motors simultaneously — for a four-wheel rover, two L298N modules are used. It also has a built-in 5V regulator (if the motor supply is 7V+) that can power the ESP32.

DC Motors

Typically TT gear motors (yellow hobby motors) or N20 micro gear motors. These are 3–6V brushed DC motors with a gear reduction box (1:48 or 1:120 ratio) that reduce speed but multiply torque, making them suitable for carrying the rover chassis and sensors. Direction is controlled by reversing polarity via the H-bridge; speed is controlled by PWM duty cycle (0–255).

MQ2 Gas Sensor

The MQ2 is a chemiresistive gas sensor with a tin dioxide (SnO₂) sensing element. Its resistance drops when it contacts combustible gases — it can detect LPG, propane, methane, smoke, hydrogen, and CO. It outputs both an analog voltage (proportional to gas concentration, read on ESP32's ADC pin) and a digital signal (HIGH when concentration exceeds a threshold set by the onboard potentiometer). The analog value is mapped to a PPM (parts per million) concentration using a calibration curve. It requires a 5V supply and needs a 24–48 hour preheat period for accurate readings.

DHT11 Sensor

The DHT11 is a low-cost digital temperature and humidity sensor. It uses a capacitive humidity sensor and a thermistor, outputting data as a single-wire serial protocol — 40 bits of data per reading (16 bits humidity + 16 bits temperature + 8 bits checksum). Its range is 0–50°C temperature (±2°C accuracy) and 20–90% relative humidity (±5% accuracy). The ESP32 reads it using the DHT library. It can only be polled once every 2 seconds. Being a digital sensor, it is immune to noise on long wire runs.

GPS Module (NEO-6M / NEO-7M)

The GPS module (typically u-blox NEO-6M) uses the UART protocol to send NMEA 0183 sentences (like \$GPRMC, \$GPGGA) to the ESP32 at 9600 baud. The ESP32 parses these sentences using the TinyGPS++ library to extract latitude, longitude, altitude, speed, and time. The module has an onboard patch antenna and a backup battery to maintain almanac data. Cold start (first fix) takes 30–60 seconds; warm start takes 1–5 seconds. The extracted coordinates are included in the JSON sensor payload sent to the Flutter app.

LED

A standard 5mm or SMD LED connected to a GPIO pin through a current-limiting resistor (typically 220Ω). It is used as a status indicator (connection established, gas alert) or as a headlight that the user can toggle from the app. The ESP32 GPIO can source/sink enough current for a single LED directly.

Power Supply

A 3S LiPo battery (7.4V nominal) or two 18650 lithium cells in series. The motor driver draws power directly from the battery for the motors, and steps down 5V for the ESP32 and sensors through the L298N's onboard regulator or a separate LM7805/buck converter module. The ESP32 itself needs a clean 3.3V (provided by its onboard AMS1117 LDO). Keeping motor power and logic power separate (or using decoupling capacitors) is critical to prevent voltage spikes from motors crashing the ESP32.

CODE

```
#include <WiFi.h>
#include <WebServer.h>
#include <DHT.h>
#include <TinyGPS++.h>

// ----- WIFI -----
const char* ssid = "AA";
const char* password = "abcd1234";

// ----- SERVER -----
WebServer server(80);

// ----- PINS -----
#define MQ2_PIN 34
#define DHT_PIN 4
#define DHTTYPE DHT11
#define IN1 13
#define IN2 26
#define IN3 14
#define IN4 27
#define LED_PIN 2

// ----- OBJECTS -----
DHT dht(DHT_PIN, DHTTYPE);
TinyGPSPlus gps;
HardwareSerial gpsSerial(2);

// ----- VARIABLES -----
bool ledState = false;

// MQ2 smoothing
int readMQ2() {
    long sum = 0;
    for (int i = 0; i < 10; i++) {
        sum += analogRead(MQ2_PIN);
        delay(2);
    }
    return sum / 10;
}

// ----- MOTOR CONTROL -----
void stopMotors() {
    digitalWrite(IN1, LOW); digitalWrite(IN2, LOW);
    digitalWrite(IN3, LOW); digitalWrite(IN4, LOW);
}

void moveForward() {
```

```
digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);
}

void moveBackward() {
digitalWrite(IN1, LOW); digitalWrite(IN2, HIGH);
digitalWrite(IN3, LOW); digitalWrite(IN4, HIGH);
}

void moveLeft() {
digitalWrite(IN1, LOW); digitalWrite(IN2, HIGH);
digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);
}

void moveRight() {
digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
digitalWrite(IN3, LOW); digitalWrite(IN4, HIGH);
}

// ----- API HANDLERS -----
// /move?dir=forward
void handleMove() {
if (!server.hasArg("dir")) {
server.send(400, "text/plain", "Missing dir");
return;
}

String dir = server.arg("dir");
if (dir == "forward") moveForward();
else if (dir == "backward") moveBackward();
else if (dir == "left") moveLeft();
else if (dir == "right") moveRight();
server.send(200, "text/plain", "OK");
}

// /stop
void handleStop() {
stopMotors();
server.send(200, "text/plain", "STOP");
}

// /led?state=on/off
void handleLED() {
if (!server.hasArg("state")) {
server.send(400, "text/plain", "Missing state");
return;
}

String state = server.arg("state");
if (state == "on") { digitalWrite(LED_PIN, HIGH); ledState = true; }
else { digitalWrite(LED_PIN, LOW); ledState = false; }
```

```

server.send(200, "text/plain", "OK");
}

// /data
void handleData() {
float temp = dht.readTemperature();
float hum = dht.readHumidity();
int gas = readMQ2();
double lat = 0.0, lon = 0.0;
if (gps.location.isValid() && gps.satellites.value() >= 4) {
lat = gps.location.lat();
lon = gps.location.lng();
}
String json = "{";
json += "\"temperature\": " + String(temp) + ",";
json += "\"humidity\": " + String(hum) + ",";
json += "\"gas\": " + String(gas) + ",";
json += "\"lat\": " + String(lat, 6) + ",";
json += "\"lon\": " + String(lon, 6);
json += "}";
server.send(200, "application/json", json);
}

// ----- SETUP -----
void setup() {
Serial.begin(115200);
pinMode(IN1,OUTPUT); pinMode(IN2,OUTPUT);
pinMode(IN3,OUTPUT); pinMode(IN4,OUTPUT);
pinMode(LED_PIN,OUTPUT);
stopMotors();
dht.begin();
gpsSerial.begin(9600, SERIAL_8N1, 16, 17);
WiFi.begin(ssid, password);
while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
Serial.println("\nConnected!");
Serial.println(WiFi.localIP());
server.on("/move", handleMove);
server.on("/stop", handleStop);
server.on("/led", handleLED);
server.on("/data", handleData);
server.begin();
}

// ----- LOOP -----
void loop() {
server.handleClient();
}

```

```
while (gpsSerial.available()) gps.encode(gpsSerial.read());  
}
```

APPLICATION CODE:

https://drive.google.com/file/d/1RgNSNTpqldBRLC1alw9KeJS_kO5XfEH/view?usp=drivesdk

SOFTWARE COMPONENTS

Flutter (Mobile App)

Flutter is Google's cross-platform UI framework using the Dart language. For this project it handles the joystick input widget (using packages like `flutter_joystick`), HTTP communication (`http` package), JSON parsing (`dart:convert`), video display (WebView or MJPEG viewer), and a real-time dashboard showing sensor data. Flutter's `setState` or a state management solution like `Provider` is used to reactively update the UI when new sensor data arrives. The app connects to the ESP32 by its static IP address on the local network.

Arduino IDE (Development Environment)

The Arduino IDE is used to write, compile, and flash firmware to both the ESP32 and ESP32-CAM. It uses the ESP32 Arduino core (by Espressif) which provides WiFi, WebServer, and peripheral libraries. The IDE handles board selection, serial monitor (for debugging), and manages library dependencies. PlatformIO (VS Code extension) is a more advanced alternative that offers better project management and faster compilation.

Embedded C (Firmware)

The firmware runs on both ESP32 boards. The main ESP32 firmware sets up a WiFi station connection, starts the HTTP web server, registers URL handlers (`/move`, `/sensors`, `/led`), and in the main loop continuously polls sensors and services HTTP clients. The ESP32-CAM firmware initializes the camera at a chosen resolution (e.g., VGA 640×480 for low latency), configures it as an MJPEG stream server, and handles concurrent viewer connections. Key libraries used include `WiFi.h`, `WebServer.h`, `DHT.h`, `TinyGPS++.h`, and `ArduinoJson.h`.

HTTP Protocol

HTTP/1.1 is used as the application-layer communication protocol over TCP/IP. The rover exposes a simple REST-like API. Control commands are sent as GET requests with query parameters (keeping them stateless and easy to debug). Sensor data is returned as JSON in the response body with `Content-Type: application/json`. The MJPEG stream uses a special `multipart/x-mixed-replace` content type where each frame is sent as a JPEG boundary, enabling continuous streaming over a single long-lived HTTP connection.

JSON Communication

`ArduinoJson` (v6/v7) on the ESP32 side and `dart:convert` on the Flutter side handle JSON serialization and deserialization. The ESP32 builds a `JsonDocument`, populates it with sensor readings, and serializes it to the HTTP response buffer. Flutter decodes the response body string into a `Map` and extracts values to update widget state. JSON is chosen for its human-readability, ease of debugging via browser, and wide library support on both platforms.

FEATURES

Joystick Control

The virtual joystick in the Flutter app captures the user's finger position and maps the X/Y displacement into four motion states — forward, reverse, left, and right — with variable speed. The displacement magnitude (0–100%) is converted into a PWM duty cycle value (0–255) and packed into an HTTP GET request sent to the ESP32's IP address. The ESP32 receives and parses the URL parameters and writes the corresponding IN1/IN2 direction signals and EN PWM signal to the motor driver. The joystick auto-centers on release, triggering a stop command immediately.

Live Video

The ESP32-CAM runs independent camera server firmware that continuously captures JPEG frames from the OV2640 camera and pushes them over WiFi using the MJPEG (Motion JPEG) protocol on a separate IP address (port 81). Each frame is sent as a boundary in a multipart HTTP response — the Flutter app keeps the HTTP connection open and renders each arriving JPEG in the video widget. Resolution is configurable (QQVGA to UXGA); VGA (640×480) at ~15 fps gives a good balance of quality and bandwidth. Because the video uses its own connection, it does not interfere with the control or sensor channels.

Sensor Monitoring

The ESP32 reads three sensor types on a continuous polling cycle. MQ2 is read via an ADC pin — the raw 12-bit value (0–4095) is converted to a voltage and mapped to a PPM concentration using the sensor's characteristic curve. DHT11 is polled every 2 seconds via a single GPIO pin using the DHT library's proprietary 1-wire protocol, returning temperature (°C) and relative humidity (%). All readings are serialized using ArduinoJson into a single JSON payload (e.g. {"temp":31,"hum":62,"gas":340,"gasppm":120}) and returned as the HTTP response body to the Flutter app's periodic GET /sensors request. The app displays these on live gauges and can trigger on-screen alerts when gas PPM exceeds a threshold.

GPS Tracking

The NEO-6M GPS module communicates with the ESP32 via UART at 9600 baud, transmitting standard NMEA 0183 sentences every second. The ESP32 reads these sentences using the TinyGPS++ library, which parses latitude, longitude, altitude, speed over ground, and UTC time from the \$GPRMC and \$GPGLA sentence types. The extracted coordinates are included in the JSON sensor payload. The Flutter app receives them and can plot the rover's position on a map widget (using flutter_map or Google Maps) in real time. The GPS module has an onboard LiPo-backed RTC so it remembers the last fix — warm start typically achieves a new fix in 1–5 seconds.

LED Control

One or more LEDs are connected to ESP32 GPIO pins via current-limiting resistors (220Ω). The ESP32 registers a dedicated HTTP endpoint (GET /led?state=on or off) so the Flutter app's toggle switch can control the LED remotely. LEDs serve a dual purpose — as manually-controlled headlights for navigating in low light (visible in the live video feed), and as automatic status indicators that the firmware controls directly. For example, the firmware can flash an LED rapidly when the MQ2 gas reading exceeds a danger threshold, giving both a visual alert on the rover and a notification in the app. PWM can also be applied to an LED pin for brightness control.

Emergency Stop

The emergency stop is the highest-priority command in the system. A dedicated large red button in the Flutter app sends GET /stop to the ESP32, which has this route registered with the highest handler priority. The ESP32 immediately sets all motor PWM enable pins to 0 and both direction pins LOW — this cuts power to all motors simultaneously regardless of any pending movement commands. The firmware also triggers emergency stop automatically if the WiFi connection is lost for more than ~500ms (watchdog timer), if the gas sensor reads above the danger threshold, or if a hardware button on the rover chassis itself is pressed. The stop state persists until an explicit resume or new movement command is received.

APPLICATIONS

- **Surveillance and Security:** Used for monitoring restricted or sensitive areas with live video streaming and remote control.
 - **Disaster Management:** Helps in search and rescue operations in hazardous environments such as collapsed buildings, fire zones, or flood-affected areas.
 - **Industrial Inspection:** Useful for inspecting pipelines, machinery, and confined spaces where human access is difficult or unsafe.
 - **Environmental Monitoring:** Collects real-time data like temperature, humidity, and gas levels for pollution and climate studies.
 - **Military and Defense:** Can be deployed for reconnaissance missions, border surveillance, and detection of dangerous conditions.
 - **Smart Agriculture:** Assists in monitoring field conditions such as soil environment and detecting harmful gases or weather changes.
 - **Remote Exploration:** Suitable for exploring unknown or dangerous terrains without risking human life.
 - **Educational and Research Purposes:** Acts as a practical platform for learning robotics, IoT, and embedded systems.
 - **Home Automation and Assistance:** Can be adapted for smart home monitoring and remote inspection tasks.
-

ADVANTAGES

- **Real-time Control:** The rover can be controlled instantly using a mobile app with low latency, ensuring smooth and responsive operation.
 - **Live Video Streaming:** Provides real-time visual feedback, improving navigation and making it suitable for surveillance and monitoring tasks.
 - **Multi-Sensor Integration:** Equipped with sensors like gas, temperature, humidity, and GPS, enabling comprehensive environmental monitoring.
 - **Wireless Operation:** WiFi-based communication eliminates the need for physical connections, allowing remote operation from a safe distance.
 - **Safety Features:** Includes emergency stop, gas detection alerts, and automatic shutdown mechanisms for safe operation in hazardous environments.
 - **User-Friendly Interface:** The mobile app offers an intuitive joystick control, live data display, and easy interaction.
 - **Scalability and Flexibility:** The modular design allows easy addition of new sensors or features without major changes.
 - **Cost-Effective Design:** Uses affordable components like ESP32 and standard sensors, making it suitable for academic and practical applications.
-

- **Energy Efficient:** Optimized communication and control reduce unnecessary power consumption.
 - **Wide Applications:** Can be used in surveillance, disaster management, industrial inspection, military operations, and environmental monitoring.
-

LIMITATIONS

- **Limited Range:** The rover operates only within the coverage of a WiFi network, restricting long-distance usage.
- **Network Dependency:** Performance depends on WiFi signal strength and stability; weak signals can cause delays or disconnection.
- **Power Constraints:** Battery life limits operation time, especially with continuous video streaming and multiple sensors.
- **Latency Issues:** Although low, some delay (20–50 ms or more) can affect precise control in critical situations.
- **Environmental Sensitivity:** Sensors like MQ2 and DHT11 may give inaccurate readings due to noise, calibration issues, or environmental factors.
- **Processing Limitations:** The ESP32 has limited processing power, which can affect multitasking under heavy loads (video, sensors, control).
- **Video Quality Trade-off:** Higher video resolution increases bandwidth usage and may reduce frame rate or cause lag.
- **GPS Accuracy:** GPS may have reduced accuracy or slow response in indoor or obstructed environments.
- **Security Concerns:** Basic HTTP communication over WiFi may be vulnerable to unauthorized access if not secured properly.
- **Mechanical Constraints:** Terrain limitations and motor capability may restrict movement on rough or uneven surfaces.

FUTURE SCOPE

- **Integration with AI and Machine Learning:** The rover can be enhanced with object detection, path planning, and autonomous navigation using AI algorithms.
 - **Cloud Connectivity (IoT):** Sensor data can be stored and analyzed on cloud platforms for remote access, data logging, and predictive analysis.
 - **Long-Range Communication:** WiFi can be replaced or supplemented with technologies like LoRa, 4G/5G, or RF modules to increase operational range.
 - **Improved Camera System:** Upgrading to HD cameras with night vision or thermal imaging can enhance surveillance capabilities.
 - **Autonomous Navigation:** Implementation of obstacle avoidance and self-navigation without continuous user control.
 - **Enhanced Security:** Adding encryption (HTTPS), authentication, and secure communication protocols to prevent unauthorized access.
 - **Extended Battery Life:** Use of high-capacity batteries or solar charging systems to increase operational time.
 - **Advanced Sensors:** Integration of additional sensors such as IR, LiDAR, or ultrasonic arrays for better environmental understanding.
 - **Robotic Arm Integration:** Adding a robotic arm for object handling, rescue operations, or industrial tasks.
 - **Mobile App Improvements:** Development of a more advanced UI with features like voice control, map tracking, and analytics dashboard.
-

CONCLUSION

The WiFi-based multi-sensor rover with real-time video streaming and mobile control interface successfully demonstrates the integration of robotics, embedded systems, and IoT technologies. The system enables efficient remote operation through a user-friendly mobile application, allowing real-time control, live video monitoring, and continuous sensor data acquisition. The use of parallel communication channels ensures low latency, reliable performance, and smooth coordination between control, sensing, and video streaming.

The rover's ability to monitor environmental conditions such as temperature, humidity, gas levels, and location makes it suitable for applications in surveillance, disaster management, and industrial inspection. Safety features like emergency stop and alert mechanisms further enhance its reliability in critical situations. Although there are certain limitations such as dependence on WiFi range and power constraints, the system provides a solid foundation for further advancements.

Overall, this project highlights a practical and cost-effective approach to developing an intelligent robotic platform, with significant potential for future improvements and real-world applications.